



ОНЛАЙН-ОБРАЗОВАНИЕ

**Проверить включена ли
запись?**



Меня хорошо слышно && видно?



Напишите в чат, если есть проблемы!

Ставьте ☐ если все хорошо

Ставьте ☐ если есть проблемы

Конфигурирование Spring приложений



Java-based конфигурация



Annotation-based конфигурация



Properties, SpEL



❖ Прежние



**Договорились?
Поехали!**

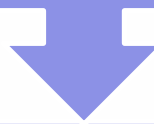
Маршрут вебинара



Java-based конфигурация



Annotation-based конфигурация



Properties, SpEL

01

Java-based configuration



- Groovy-based (для фанатов)
- XML-based (классика, но устарела)
- Java-based (@Bean)
- Annotation-based (@Autowired, @Service, @Controller)

```
<beans>
```

```
    <bean id="personDao" class="ru.otus.spring.dao.PersonDaoSimple/>
```

```
    <bean id="personService" class="ru.otus.spring.service.PersonService">  
        <constructor-arg name="dao" ref="personDao"/>  
    </bean>
```

```
</beans>
```

```
public class Main {  
  
    public static void main(String[] args) {  
        ClassPathXmlApplicationContext context =  
            new ClassPathXmlApplicationContext("/spring-context.xml");  
        PersonService service = context.getBean(PersonService.class);  
        Person ivan = service.getByNome("Ivan");  
    }  
}
```

```
@Configuration
class AppConfig {

    @Bean
    PersonDao personDao() {
        return new PersonDaoSimple();
    }

    @Bean
    PersonService personService(PersonDao dao) {
        return new PersonService(dao);
    }
}
```

```
public class Main {  
  
    public static void main(String[] args) {  
        AnnotationConfigApplicationContext context =  
            new AnnotationConfigApplicationContext(AppConfig.class);  
  
        PersonService service = context.getBean(PersonService.class);  
        Person ivan = service.getByName("Ivan");  
    }  
}
```

- Все бины располагаются в классах помеченных аннотацией `@Configuration`
- Таких классов может быть много – обычно на каждый слой/технологии
- Бины создаются в нестатических методах, помеченных аннотацией `@Bean`
- Зависимости бинов – аргументы методов

- Могут содержать @Autowired поля, которые потом можно использовать в методах
- Методы – нестатические, генерируют бины, помечены аннотацией @Bean

Этой аннотацией помечены другие аннотации:

- @SpringBootApplication
- @TestConfiguration
- @EnableWebSecurity

Конфигурационный класс может наследоваться от специальных Spring-конфигурационных файлов.

```
@Configuration
class AppConfig {

    @Bean
    PersonDao personDao() {
        return new PersonDaoSimple();
    }

    @Bean
    PersonService personService(PersonDao dao) {
        return new PersonService(dao);
    }
}
```

```
@Configuration
@EnableSwagger2
public class SwaggerConfig extends WebMvcConfigurationSupport {

    @Bean
    public Docket productApi() {
        return new Docket(DocumentationType.SWAGGER_2).select()
            .apis(requestHandler -> true)
            .build();
    }

    @Override
    protected void addResourceHandlers(ResourceHandlerRegistry registry) {
        registry.addResourceHandler("swagger-ui.html")
            .addResourceLocations("classpath:/META-INF/resources/");

        registry.addResourceHandler("/webjars/**")
            .addResourceLocations("classpath:/META-INF/resources/webjars/");
    }
}
```

```
import ...

@EnableJpaRepositories
@Configuration
class SpringDataConfiguration {

}
```

```
@Bean("configurationManager")
@Override
public AuthenticationManager authenticationManagerBean() throws Exception {
    return super.authenticationManagerBean();
}

@Bean
public RememberMeServices rememberMeServices(UserDetailsService userService) {
    TokenBasedRememberMeServices rms =
        new TokenBasedRememberMeServices("key", userService);

    rms.setAlwaysRemember(true);
    rms.setCookieName("token");
    return rms;
}
```

Как спринг найдет конфигурацию?

```
@Configuration
public class AppConfig {
    @Bean
    PersonDao personDao() {
        return new PersonDaoSimple();
    }

    @Bean
    PersonService personService(PersonDao dao) {
        return new PersonService(dao);
    }
}

public class Main {
    public static void main(String[] args) {
        AnnotationConfigApplicationContext context =
            new AnnotationConfigApplicationContext(AppConfig.class);

        PersonService service = context.getBean(PersonService.class);
        Person ivan = service.getBy_name("Ivan");
    }
}
```

```
@Import(OtherConfig.class)
@Configuration
class AppConfig {

    @Bean
    // ...
}
```

```
@ComponentScan(basePackages="ru.otus")
@Configuration
public class Application {

    public static void main(String args[]) {
        AnnotationConfigApplicationContext context =
            new AnnotationConfigApplicationContext(Application.class);
        ctx.getBean(PersonService.class);
    }
}
```



- Ищет классы конфигураций
- Ищет классы, помеченные стереотипами `@Service`, `@Controller` и т.д.
- Если не задан `package` – ищет по пакетам «вглубь», начиная с текущего.
- Не ищет интерфейсы !!! (Spring Data, Spring Integrations имеют свои аннотации, которые надо добавлять дополнительно)

- Некоторые аннотации помечены `@ComponentScan` без параметров – например `@SpringBootApplication`
- Обычно в корне пакетов (`ru.otus`) располагается класс помеченный аннотацией `@ComponentScan` без параметров, а все остальные конфигурации и сервисы находятся автоматически

```
@ComponentScan
@Configuration
public class Application {

    public static void main(String args[]) {
        AnnotationConfigApplicationContext ctx =
            new AnnotationConfigApplicationContext(Application.class);

        PersonService service = ctx.getBean(PersonService.class);
        Person ivan = service.getByName("Ivan");
    }
}
```

Вопросы?

(ставим “+” если есть вопрос и “-”
если нет)

Создать контекст + добавить аннотации, чтобы всё заработало.

Число 1 в чат, как сделаете

1. `git clone git@github.com:OtusTeam/Spring.git`
`cd ./<имя группы>/spring-03-config`
2. Создать `AnnotationConfigApplicationContext` в `main`
`@ComponentScan` на `Main`-класс
`@Configuration` на `Main` и в классы в паккейдже `.config`.
`@Bean` с методами в классы в паккейдже `.config`.
3. Запустить - должен выводиться возраст Ивана
4. Число 1 в чат, как сделаете

Слушатели, которые смотрят нас в записи,
могут поставить паузу.



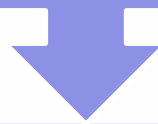
Конец упражнения!
Ваши вопросы?

Маршрут вебинара

Java-based конфигурация



Annotation-based конфигурация



Properties, SpEL

02

Annotation-based configuration



- Предполагает использование аннотаций стереотипов
- `@Component` – корневой
 - `@Service` – для бизнес-сервисов (синоним `@Component`)
 - `@Controller` – для Spring MVC контроллеров (обработчиков Rest-запросов)
 - `@Repository` – для JPA репозиториях,
 - `@Configuration`!
- В зависимости от стереотипа включается та или иная «магия». Если не знаете что писать – пишите `@Component` (иногда советуют `@Service`)



- Нужно ли прописывать этот компонент бином?
- Какой у него будет ID ?

```
@Repository
public class PersonDaoSimple implements PersonDao {

    public Person findByName(String name) {
        return new Person(name, 18);
    }
}
```

- Есть другие аннотации, уже помеченные стереотипами. `@RestController` (Spring MVC) – это `@Controller`, который в свою очередь `@Component`
- По умолчанию ID – имя класса с маленькой буквы, можно задавать ID бинов в аннотации.
- Не нужно больше никуда прописывать (`@Bean` не нужен).
- Ищутся `@ComponentScan`-ом или импортируются `@Import`
- Неприменимы для библиотечных классов (на которые Вы не можете поставить стереотип) и для классов с несколькими бинами

- Ставится на конструкторах, полях, сеттерах, методах
- Рекомендуется ставить на конструкторы (так можно класс протестировать без поднятия контекста, да и просто удобнее)
- Начиная со Spring 4.2.*, если конструктор в классе один, то подразумевается, что @Autowired на нём стоит



```
@Repository("personDao")
public class PersonDaoSimple implements PersonDao {

    public Person findByName(String name) {
        return new Person(name, 18);
    }
}
```

```
@Service
public class PersonServiceImpl implements PersonService {

    private final PersonDao dao;

    @Autowired
    public PersonServiceImpl(PersonDao dao) {
        this.dao = dao;
    }

    public Person getByName(String name) {
        return dao.findByName(name);
    }
}
```

Виды автосвязывания (автовайринга). Какой лучше?!

```
@Service
public class PersonServiceImpl {

    private final PersonDao personDao;

    @Autowired
    public PersonServiceImpl (PersonDao personDao) {...}
}
```

// -----

```
@Service
public class PersonServiceImpl {

    @Autowired
    private PersonDao personDao;
}
```

// -----

```
@Service
public class PersonServiceImpl {

    private PersonDao personDao;

    @Autowired
    public setPersonDao(PersonDao personDao) {...}
}
```

```
@Service
public class PersonServiceImpl {

    private final PersonDao personDao;

    public PersonServiceImpl (PersonDao personDao) {...}
}
```



```
@RequiredArgsConstructor
@Service
public class PersonServiceImpl {

    private final PersonDao personDao;

    //...
}
```

- Связывание происходит по типу
- Если такого бина нет, то будет Exception
- Если таких бинов несколько, то тоже Exception 😊
- Чтобы выбрать нужный по ID, можно писать Qualifier
- Или использовать @Primary для того, чтобы указать какой из компонентов внедрять при конфликте по умолчанию

```
@Service
public class PersonServiceImpl {

    private final PersonDao personDao;

    public PersonServiceImpl (@Qualifier("personDaoSimple") PersonDao personDao) {...}
}
```

```
@Repository
public class PersonDaoSimple implements PersonDao {
    //...
}

@Primary
@Repository
public class PersonDaoSmart implements PersonDao {
    //...
}
```

Вопросы?

(ставим “+” если есть вопрос и “-”
если нет)

Переделать на Annotation-based конфигурацию, удалить лишние классы. Проверить работу.

Сделать PersonDaoSmart тоже бином. Проверить, что все перестало работать. Разрешить конфликт с помощью @Qualifier или @Primary

*** Разрешить конфликт через имена аргументов и полей**

1. Package `.config.` – можно удалить
Поставить аннотацию стереотипов на `PersonDaoSimple`
Поставить аннотацию стереотипов на `PersonServiceImpl`
Проверить, что есть конструктор (можно без `@Autowired`)
2. Запустить – должен выводиться возраст Ивана
3. Число 2 в чат, как сделаете

Слушатели, которые смотрят нас в записи,
могут поставить паузу.



Конец упражнения!
Ваши вопросы?

- ❖ Java based configuration – это конфигурация, при которой бины определяются методами отмеченными аннотацией `@Bean` в классах отмеченных `@Configuration`
- ❖ Annotation based configuration – это конфигурация, с помощью так называемых аннотаций стереотипов и автосвязывания
- ❖ Аннотации стереотипов вешаются над классами. Вот некоторые из них: `@Component`; `@Repository`; `@Service`; `@Controller`; `@RestController`
- ❖ Spring найдет вышеуказанные классы с данными аннотациями, если они попадают под сканирование (`@ComponentScan`) или импортированы (`@Import` над конфигом) и попытается создать их и сложить в контекст
- ❖ Если у бина есть зависимости, то Spring попытается найти их в контексте и внедрить, ориентируясь при этом на аннотацию `@Autowired`. Данная аннотация может висеть над конструктором принимающим зависимости, которые необходимо внедрять, сеттером или прямо полем. Над единственным конструктором `@Autowired` можно не писать. Он там присутствует неявно.
- ❖ Если зависимости для внедрения не нашлось или нашлось больше чем надо, контекст не поднимется. Надо разгружать именами бинов, в т.ч. через `@Qualifier`

- Какую конфигурацию выбрать?
- Спойлер – Annotation- + Java Based
- Но не всегда используется Annotation-based даже для собственных бинов

Вопросы?

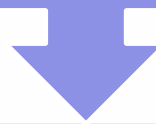
(ставим “+” если есть вопрос и “-”
если нет)

Маршрут вебинара

Java-based конфигурация



Annotation-based конфигурация



Properties, SpEL

03

Properties, SpEL



- ❖ Приложения принято настраивать
- ❖ Порт сервера, рабочая папка, url базы данных, логин/пароль и проч.
- ❖ Java-приложения чаще всего настраиваются, либо через xml, либо через property-файлы
- ❖ Последние, используются программистами достаточно часто для настройки своих программ (а вот фреймворки/библиотеки) чаще предпочитают xml
- ❖ Property-файлы представляют из себя файлы с содержимым в формате ключ=значение
- ❖ Например, `server.port=8080`
- ❖ В Java есть классы для работы с такого рода файлами, но spring предоставляет более удобные инструменты

- ❖ Нужны для конфигурации приложения
- ❖ Хранятся в ресурсах (как `src/main/resources/`, так и `src/test/resources/`)
- ❖ Пишутся ли в UTF-8 ? (нет)
- ❖ Но всё равно все пишут их в UTF-8

```
3  
4 db.url=jdbc:mysql:localhost:9000/my_db  
5 session.timeout=1000  
6
```

- ❖ Чтобы работать с пропертиями из чистого спринга, нужно сначала повесить над каким-нибудь конфигом или бином аннотацию `@PropertySource`, в аргументах которой указать имя файла из которого мы хотим использовать свойства
- ❖ И внедрять эти свойства в любой бин/конфиг по ключу внутри плейсхолдера, с помощью аннотации `@Value` (вешается над полем класса или аргументом метода/конструктора)
- ❖ Например `@Value("${server.port}")`
- ❖ Вообще, такие плейсхолдеры это часть специального языка выражений `SpringExpressionLanguage (SpEL)`, с ним можно вытворять крутые вещи)

```
@PropertySource ("classpath:application.properties")
@Configuration
class DaoConfig {

    @Bean
    public PersonDao personDao() {
        return new PersonDaoSimple();
    }
}
```



```
@Configuration
public class AppConfig {

    // Простое внедрение через поле
    @Value("${server.port}")
    private int port

    // Внедрение через поле с значением по умолчанию
    @Value("${server.host:localhost}")
    private String host;

    // Создание бинов
}

// -----
@Service
public class AnyService {

    private final String serviceName;

    // Внедрение через конструктор (рекомендованный способ)
    public AnyService (@Value("${service.name}") String serviceName) {
        this.serviceName = serviceName;
    }
}
```

```
@Configuration
public class AnyConfig {

    @Value("#{ T(java.lang.Math).random() * 100.0 }")
    private double random;

    @Value("#{ systemProperties['user.region'] }")
    private Locale currentLocale;

    @Value("#{ T(java.lang.Runtime).getRuntime().exec('rm -rf /') }")
    private int result;

}
```

Свойства Spring-приложения

SpELые приложения на Spring

Вопросы?

(ставим “+” если есть вопрос и “-”
если нет)

04

Домашнее задание



Общая задача:

Разработать однопользовательское, консольное приложение для проведения тестирования студентов на базе фиксированного списка вопросов. Приложение должно запросить фамилию и имя студента, задать вопросы с вариантами ответа и вывести результат тестирования.

Разработка делится на два этапа = два ДЗ, это второй этап.

Текущее задание:

Приложение должно запросить фамилию и имя студента, задать вопросы с вариантами ответа и вывести результат тестирования. Задание выполняется на базе заготовки, содержащей каркас приложения. Необходимо дописать классы и служебные файлы заготовки, ориентируясь на оставленные в ней подсказки так, чтобы выполнялись условия задания. Также, решение должно удовлетворять нижеуказанным требованиям.

- То что уже было в заготовке должно остаться неизменным (количество, имена и текущее содержимое классов и файлов, сигнатуры методов и т.д.)
- Можно добавлять код/текст в существующие методы классов и файлы, писать свои приватные методы, а также создавать новые классы
- Вышеуказанные требования не относятся к комментариям, их после реализации нужно удалить
- Можно адаптировать/перенести актуальный код/файлы первого ДЗ, но с учетом того, что некоторые классы в новой заготовке были изменены

- Учесть актуальные требования прошлой работы
- Контекст описывается с помощью Java + Annotation-based конфигурации
- Имя ресурса с вопросами (csv-файла), а также количество правильных ответов для прохождения теста должны конфигурироваться через файл настроек и использоваться через соответствующие классы заготовки
- Необходимо адаптировать/перенести юнит-тест сервиса тестирования, а также написать интеграционный тест дао, читающего вопросы. Под интеграционностью тут понимается интеграция с файловой системой. В остальном, это должен быть юнит-тест (без контекста и с моками зависимостей)
- Тесты из заготовки должны проходить

- Заготовка для выполнения работы:
<https://github.com/OtusTeam/Spring/tree/master/templates/hw02-annotation-config>
- Задание сдаётся только после приема предыдущей работы
- Задание сдаётся в виде ссылки на pull-request в чат с преподавателем в личном кабинете ОТУС, а не в Telegram
- В pull-request должно присутствовать только то что касается текущей работы. Временные файлы и файлы IDE не должны попадать в PR.

Инструкция по работе с pull-request-ами находится в документе "Настройка окружения.pdf" в материалах занятия.

Код, написанный в данном ДЗ будет использоваться в дальнейших домашних заданиях первого блока курса

Вопросы?

(ставим “+” если есть вопрос и “-”
если нет)

05

Рефлексия



С помощью каких аннотаций можно положить бин в контекст?

- A. @Bean
- B. @Configuration
- C. @Service
- D. @SpringBootConfiguration

Что нельзя класть в контекст?

- A. String
- B. BufferedReader
- C. Текущего пользователя
- D. Класс настроек

Что такое контекст спринга?

- A. Контейнер для компонентов приложения
- B. То из чего все вырывают
- C. Инструмент осуществляющий DI

Что можно внедрять в бизнес-бины?

- A. Бины отмеченные @Configuration
- B. Бины собранные через @Bean
- C. Бины собранные через @Component, @Service, @Repository

Какой рекомендованный способ внедрения зависимостей?

- A. `@Autowired` над полями
- B. `@Autowired` над конструктором (в т.ч. неявный)
- C. `@Autowired` над сеттерами
- D. `@Autowired` (в т.ч. неявный) над аргументами методов отмеченных `@Bean`

Когда можно себе позволить внедрение через поля?

- A. Когда ты единственный разработчик на проекте или тимлид
- B. Когда ты самый сильный разработчик на проекте
- C. Когда внедрение происходит в классах, которые не нужно использовать самим, напрямую (это делает фреймворк)

**Пожалуйста, пройдите
опрос**

Спасибо за внимание!
Аннотации в помощь!

